Low Level Design (LLD)

Hisab Kitab — Personal Finance Ledger System

Document Version: 1.0
Author: [Your Name]
Date: June 2025
Depends on: SRS v1.0, Tech Stack Selection v1.0

---

1. What Is a Low Level Design and Why Does It Exist?

If the SRS answers *what* the system must do, the LLD answers *how* the code is structured to do it. It is the bridge between a requirements document and actual source code. Writing an LLD before coding forces you to think through class responsibilities, method signatures, and interaction sequences before you are committed to any implementation. Changes in a design document cost nothing — changes in running code cost time and introduce bugs.

A well-written LLD also serves as onboarding documentation. If a new developer joins the project (or if you return to it after six months), the LLD tells them exactly where each piece of logic lives and how the layers talk to each other.

---

2. Project Directory Structure

Understanding the folder structure is a prerequisite to understanding the class design, because the folder structure *is* the architecture made visible.

```
hisab-kitab/
├── backend/
│   ├── app/
│   │   ├── main.py                 # FastAPI app instantiation, router
registration
│   │   ├── config.py               # Settings (read from .env using pydantic-
settings)
│   │   ├── database.py             # SQLAlchemy engine, session factory, Base
class
│   │   │
│   │   ├── models/                 # SQLAlchemy ORM classes (database tables)
│   │   │   ├── __init__.py
│   │   │   ├── base.py             # Abstract BaseModel (id, created_at,
updated_at)
│   │   │   ├── user.py             # User model
│   │   │   ├── party.py            # Party model
│   │   │   ├── khata.py            # Khata model + KhataType + KhataStatus enums
│   │   │   └── entry.py            # Entry model + EntryDirection enum
│   │   │
│   │   ├── schemas/                # Pydantic models (API request/response
shapes)
│   │   │   ├── auth.py             # LoginRequest, TokenResponse
│   │   │   ├── party.py            # PartyCreate, PartyResponse
│   │   │   ├── khata.py            # KhataCreate, KhataResponse,
KhataWithBalance
│   │   │   ├── entry.py            # EntryCreate, EntryResponse,
EntryWithBalance
│   │   │   └── report.py           # MonthlySummaryResponse, OutstandingResponse
│   │   │
│   │   ├── repositories/           # Database query layer
│   │   │   ├── base.py             # BaseRepository (generic CRUD)
│   │   │   ├── user_repo.py        # UserRepository
│   │   │   ├── party_repo.py       # PartyRepository
│   │   │   ├── khata_repo.py       # KhataRepository
│   │   │   └── entry_repo.py       # EntryRepository
│   │   │
│   │   ├── services/               # Business logic layer
│   │   │   ├── auth_service.py     # AuthService (login, token creation)
│   │   │   ├── khata_service.py    # KhataService (create, settle, balance)
│   │   │   ├── entry_service.py    # EntryService (add, delete, running balance)
│   │   │   └── report_service.py   # ReportService (monthly summary,
outstanding)
│   │   │
```

```
|   |   ├── api/
|   |   |   └── v1/
|   |   |       ├── auth.py          # POST /auth/login, POST /auth/logout
|   |   |       ├── parties.py       # CRUD endpoints for Party
|   |   |       ├── khatas.py        # CRUD endpoints for Khata
|   |   |       ├── entries.py       # CRUD endpoints for Entry
|   |   |       └── reports.py       # GET /reports/monthly, GET
/reports/outstanding
|   |   |
|   |   └── dependencies.py          # FastAPI dependency injection
(get_current_user, get_db)
|   |
|   ├── tests/
|   |   ├── test_khata_service.py
|   |   ├── test_entry_service.py
|   |   └── test_report_service.py
|   |
|   ├── alembic/                     # Database migration files
|   ├── requirements.txt
|   └── .env.example
|
└── frontend/
    ├── src/
    |   ├── api/                     # All Axios calls centralized here
    |   |   ├── client.js            # Axios instance with base URL and auth
headers
    |   |   ├── khataApi.js
    |   |   ├── entryApi.js
    |   |   └── reportApi.js
    |   ├── pages/
    |   |   ├── Home.jsx             # Khata list (home screen)
    |   |   ├── KhataDetail.jsx      # Entry list for one Khata
    |   |   ├── AddEntry.jsx         # Add entry form
    |   |   └── Report.jsx           # Monthly summary
    |   ├── components/
    |   |   ├── KhataCard.jsx
    |   |   ├── EntryRow.jsx
    |   |   └── BalanceBadge.jsx
    |   └── store/                   # Zustand stores
    └── public/
        └── manifest.json            # PWA manifest
```

3. Model Layer — Class Design

All SQLAlchemy models inherit from a single `BaseModel` abstract class. This is a classic OOP pattern called the Template Pattern — the parent defines shared structure (primary key, timestamps), and children add domain-specific fields. This eliminates repetition and ensures that every table in the database has consistent auditing columns.

3.1 BaseModel

python

```python
# app/models/base.py
import uuid
from datetime import datetime
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column
from sqlalchemy import DateTime, func

class Base(DeclarativeBase):
    pass

class BaseModel(Base):
    """
    Abstract base for all ORM models.
    Every table gets: UUID primary key, created_at, updated_at.
    __abstract__ = True means SQLAlchemy will NOT create a table for this class.
    """
    __abstract__ = True

    id: Mapped[uuid.UUID] = mapped_column(
        primary_key=True,
        default=uuid.uuid4
    )
    created_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True),
        server_default=func.now()   # Set by database, not application code
    )
    updated_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True),
        server_default=func.now(),
        onupdate=func.now()         # Automatically updated on any row change
    )
```

3.2 User Model

python

```python
# app/models/user.py
from sqlalchemy.orm import Mapped, mapped_column, relationship
from sqlalchemy import String
from .base import BaseModel

class User(BaseModel):
    __tablename__ = "users"

    name: Mapped[str] = mapped_column(String(100))
    phone: Mapped[str] = mapped_column(String(15), unique=True, index=True)
    pin_hash: Mapped[str] = mapped_column(String(255))

    # Relationships (SQLAlchemy loads these lazily by default)
    khatas: Mapped[list["Khata"]] = relationship(back_populates="user")
    parties: Mapped[list["Party"]] = relationship(back_populates="user")
```

3.3 Party Model

python

```python
# app/models/party.py
from sqlalchemy.orm import Mapped, mapped_column, relationship
from sqlalchemy import String, ForeignKey
import uuid
from .base import BaseModel

class Party(BaseModel):
    __tablename__ = "parties"

    name: Mapped[str] = mapped_column(String(100))
    phone: Mapped[str | None] = mapped_column(String(15), nullable=True)
    user_id: Mapped[uuid.UUID] = mapped_column(ForeignKey("users.id"))

    user: Mapped["User"] = relationship(back_populates="parties")
    khatas: Mapped[list["Khata"]] = relationship(back_populates="party")
```

3.4 Khata Model

python

```python
# app/models/khata.py
import enum
import uuid
from sqlalchemy.orm import Mapped, mapped_column, relationship
from sqlalchemy import String, Enum, ForeignKey
from .base import BaseModel

class KhataType(str, enum.Enum):
    PERSONAL = "PERSONAL"
    EXPENSE_ACCOUNT = "EXPENSE_ACCOUNT"

class KhataStatus(str, enum.Enum):
    OPEN = "OPEN"
    SETTLED = "SETTLED"

class Khata(BaseModel):
    __tablename__ = "khatas"

    title: Mapped[str] = mapped_column(String(150))
    khata_type: Mapped[KhataType] = mapped_column(Enum(KhataType))
    status: Mapped[KhataStatus] = mapped_column(
        Enum(KhataStatus),
        default=KhataStatus.OPEN
    )
    user_id: Mapped[uuid.UUID] = mapped_column(ForeignKey("users.id"))
    party_id: Mapped[uuid.UUID | None] = mapped_column(
        ForeignKey("parties.id"),
        nullable=True   # None for EXPENSE_ACCOUNT type Khatas
    )

    user: Mapped["User"] = relationship(back_populates="khatas")
    party: Mapped["Party | None"] = relationship(back_populates="khatas")
    entries: Mapped[list["Entry"]] = relationship(
        back_populates="khata",
        order_by="Entry.date.desc()"   # Always newest first
    )
```

3.5 Entry Model

python

```python
# app/models/entry.py
import enum
import uuid
from decimal import Decimal
from datetime import date
from sqlalchemy.orm import Mapped, mapped_column, relationship
from sqlalchemy import Enum, ForeignKey, Numeric, Date, String
from .base import BaseModel

class EntryDirection(str, enum.Enum):
    JAMA = "JAMA"   # Money IN (received)
    UDHAR = "UDHAR" # Money OUT (given)

class Entry(BaseModel):
    __tablename__ = "entries"

    khata_id: Mapped[uuid.UUID] = mapped_column(ForeignKey("khatas.id"))
    date: Mapped[date] = mapped_column(Date, index=True)
    # Numeric(12, 2) = up to 9,999,999,999.99 — never use Float for money
    amount: Mapped[Decimal] = mapped_column(Numeric(12, 2))
    direction: Mapped[EntryDirection] = mapped_column(Enum(EntryDirection))
    note: Mapped[str | None] = mapped_column(String(255), nullable=True)

    khata: Mapped["Khata"] = relationship(back_populates="entries")
```

---

4. Repository Layer — Class Design

The repository layer abstracts all database queries. A service never writes SQLAlchemy query syntax — it calls a repository method with a descriptive name. This makes services testable with mock repositories and keeps SQL out of business logic.

4.1 BaseRepository

python

```python
# app/repositories/base.py
from typing import TypeVar, Generic, Type
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy import select
from app.models.base import BaseModel
import uuid

ModelType = TypeVar("ModelType", bound=BaseModel)

class BaseRepository(Generic[ModelType]):
    """
    Generic CRUD repository.
    Subclasses inject a specific model type at instantiation.

    Example:
        class KhataRepository(BaseRepository[Khata]):
            def __init__(self, session): super().__init__(Khata, session)
    """
    def __init__(self, model: Type[ModelType], session: AsyncSession):
        self.model = model
        self.session = session

    async def get_by_id(self, id: uuid.UUID) -> ModelType | None:
        result = await self.session.get(self.model, id)
        return result

    async def create(self, obj: ModelType) -> ModelType:
        self.session.add(obj)
        await self.session.flush()  # Gets the DB-generated ID without committing
        return obj

    async def delete(self, obj: ModelType) -> None:
        await self.session.delete(obj)
        await self.session.flush()
```

4.2 KhataRepository (domain-specific queries)

python

```python
# app/repositories/khata_repo.py
from sqlalchemy import select, func
from app.models.khata import Khata, KhataStatus
from app.models.entry import Entry, EntryDirection
from .base import BaseRepository
import uuid

class KhataRepository(BaseRepository[Khata]):
    def __init__(self, session):
        super().__init__(Khata, session)

    async def get_all_open(self, user_id: uuid.UUID) -> list[Khata]:
        """Returns all open Khatas for a user, sorted by absolute balance descending
        result = await self.session.execute(
            select(Khata)
            .where(Khata.user_id == user_id, Khata.status == KhataStatus.OPEN)
        )
        return result.scalars().all()

    async def get_balance(self, khata_id: uuid.UUID) -> dict:
        """
        Computes Jama total, Udhar total, and net balance in one SQL query.
        Using the database for aggregation is always faster than fetching
        all rows and summing in Python.
        """
        result = await self.session.execute(
            select(
                func.sum(
                    Entry.amount.filter(Entry.direction == EntryDirection.JAMA)
                ).label("total_jama"),
                func.sum(
                    Entry.amount.filter(Entry.direction == EntryDirection.UDHAR)
                ).label("total_udhar"),
            ).where(Entry.khata_id == khata_id)
        )
        row = result.one()
        total_jama = row.total_jama or 0
        total_udhar = row.total_udhar or 0
        return {
            "total_jama": total_jama,
            "total_udhar": total_udhar,
```

```
            "balance": total_jama - total_udhar
        }
```

---

5. Service Layer — Class Design

Services contain the business rules. They decide *whether* an operation is valid before asking the repository to execute it.

5.1 KhataService

python

```python
# app/services/khata_service.py
from app.repositories.khata_repo import KhataRepository
from app.repositories.party_repo import PartyRepository
from app.schemas.khata import KhataCreate
from app.models.khata import Khata, KhataType, KhataStatus
from fastapi import HTTPException
import uuid

class KhataService:
    def __init__(
        self,
        khata_repo: KhataRepository,
        party_repo: PartyRepository
    ):
        # Dependency injection: services receive their repos, never create them
        self.khata_repo = khata_repo
        self.party_repo = party_repo

    async def create_khata(
        self,
        data: KhataCreate,
        user_id: uuid.UUID
    ) -> Khata:
        """
        Business rule: A PERSONAL Khata must have a party_id.
        An EXPENSE_ACCOUNT Khata must NOT have a party_id.
        """
        if data.khata_type == KhataType.PERSONAL and not data.party_id:
            raise HTTPException(
                status_code=422,
                detail="A personal Khata must be linked to a party."
            )
        if data.khata_type == KhataType.EXPENSE_ACCOUNT and data.party_id:
            raise HTTPException(
                status_code=422,
                detail="An expense account Khata cannot be linked to a party."
            )
        khata = Khata(
            title=data.title,
            khata_type=data.khata_type,
            party_id=data.party_id,
            user_id=user_id
        )
```

```python
        return await self.khata_repo.create(khata)

    async def settle_khata(self, khata_id: uuid.UUID, user_id: uuid.UUID) -> Khata:
        """Business rule: Can only settle an OPEN Khata that belongs to this user."""
        khata = await self.khata_repo.get_by_id(khata_id)
        if not khata or khata.user_id != user_id:
            raise HTTPException(status_code=404, detail="Khata not found.")
        if khata.status == KhataStatus.SETTLED:
            raise HTTPException(status_code=409, detail="Khata is already settled.")
        khata.status = KhataStatus.SETTLED
        return khata
```

5.2 EntryService

python

```python
# app/services/entry_service.py
from app.repositories.khata_repo import KhataRepository
from app.repositories.entry_repo import EntryRepository
from app.schemas.entry import EntryCreate
from app.models.entry import Entry
from app.models.khata import KhataStatus
from fastapi import HTTPException
import uuid

class EntryService:
    def __init__(self, entry_repo: EntryRepository, khata_repo: KhataRepository):
        self.entry_repo = entry_repo
        self.khata_repo = khata_repo

    async def add_entry(
        self,
        khata_id: uuid.UUID,
        data: EntryCreate,
        user_id: uuid.UUID
    ) -> Entry:
        """
        Business rules:
          1. Khata must exist and belong to this user.
          2. Khata must be OPEN (cannot add entries to a settled Khata).
          3. Date must not be in the future (checked in Pydantic schema).
        """
        khata = await self.khata_repo.get_by_id(khata_id)
        if not khata or khata.user_id != user_id:
            raise HTTPException(status_code=404, detail="Khata not found.")
        if khata.status == KhataStatus.SETTLED:
            raise HTTPException(
                status_code=409,
                detail="Cannot add entries to a settled Khata. Re-open it first."
            )
        entry = Entry(
            khata_id=khata_id,
            date=data.date,
            amount=data.amount,
            direction=data.direction,
            note=data.note
        )
        return await self.entry_repo.create(entry)
```

6. API Layer — Endpoint Reference

The following table lists every API endpoint. This serves as the contract between the frontend and backend teams — in this case, both roles are filled by the same developer, but the discipline of defining the contract before implementation is valuable practice.

Method	Path	Auth	Description	FR
POST	/api/v1/auth/login	No	Login with phone + PIN, returns JWT	FR-01
POST	/api/v1/auth/logout	Yes	Client-side token clear	FR-03
GET	/api/v1/parties	Yes	List all parties	FR-05
POST	/api/v1/parties	Yes	Create a party	FR-04
GET	/api/v1/khatas	Yes	List all open Khatas with balance	FR-08, FR-09
POST	/api/v1/khatas	Yes	Create a Khata	FR-07
GET	/api/v1/khatas/{id}	Yes	Get single Khata with balance	FR-09
PATCH	/api/v1/khatas/{id}/settle	Yes	Settle a Khata	FR-10
PATCH	/api/v1/khatas/{id}/reopen	Yes	Re-open a settled Khata	FR-11
GET	/api/v1/khatas/{id}/entries	Yes	List entries for a Khata	FR-13, FR-15
POST	/api/v1/khatas/{id}/entries	Yes	Add an entry to a Khata	FR-12
DELETE	/api/v1/entries/{id}	Yes	Delete an entry	FR-14
GET	/api/v1/reports/monthly	Yes	Monthly summary (?year=&month=)	FR-16
GET	/api/v1/reports/outstanding	Yes	All outstanding balances	FR-17
GET	/api/v1/search	Yes	Global search (?q=)	FR-18

7. Sequence Diagrams

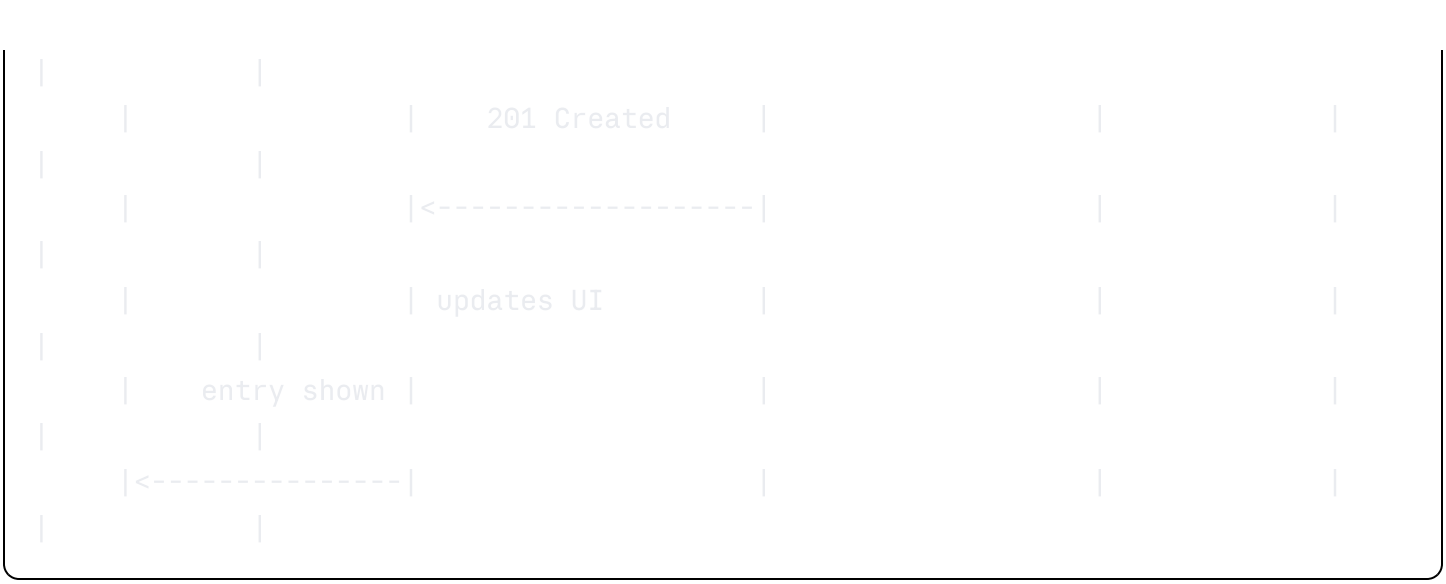
A sequence diagram shows the exact chain of calls that happen when a user performs an action. Reading these diagrams tells you the precise order of operations and which layer is responsible

for what.

7.1 Sequence: User Adds an Entry

This is the most common action in the system — the father recording what happened today.

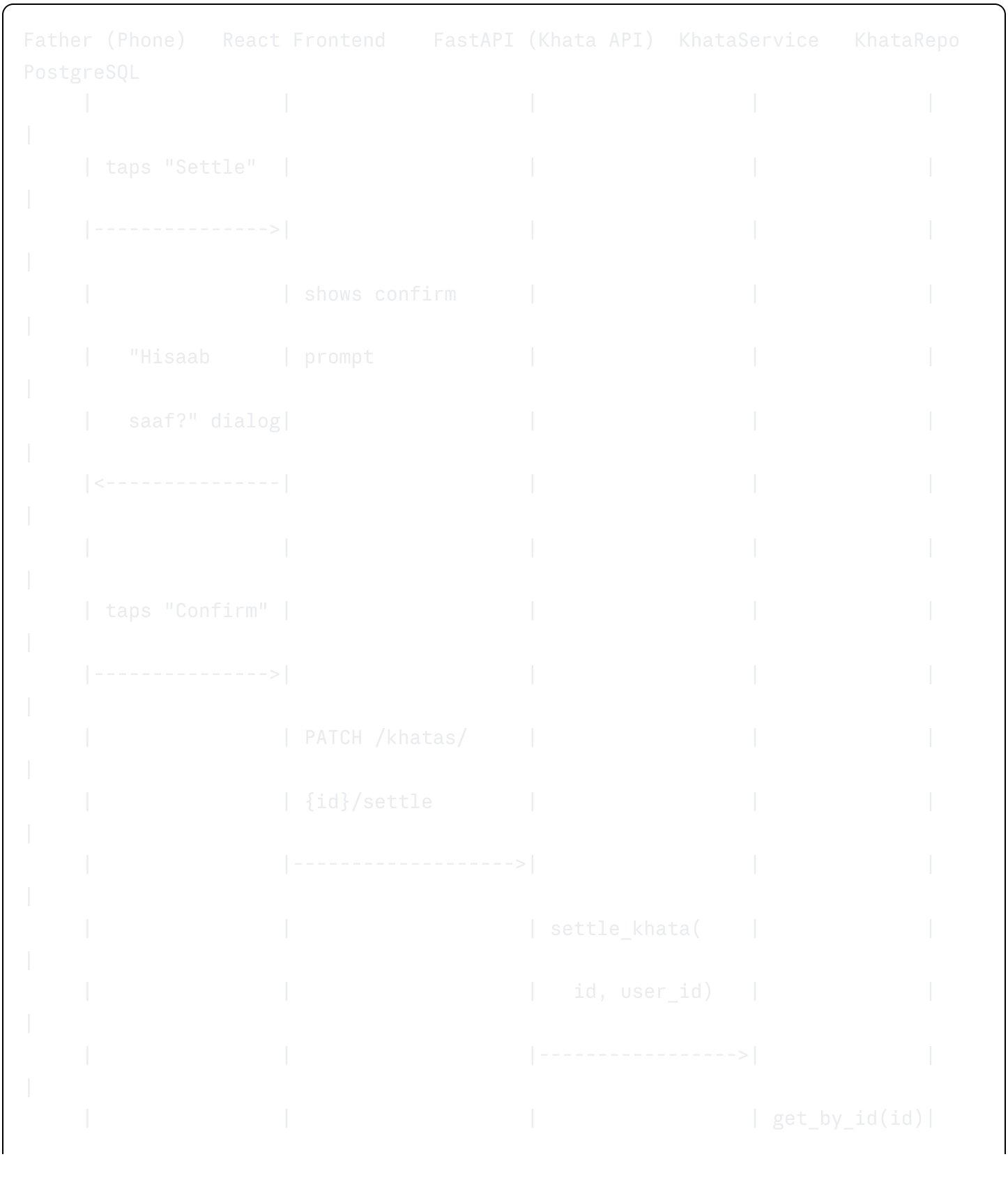

Father (Phone)
React Frontend
FastAPI (Entry API)
EntryService
KhataRepo
EntryRepo
PostgreSQL
taps "Add"
shows form
fills form,
taps "Save"
POST /khatas/{id}
/entries
validate JWT
validate schema
add_entry(
khata_id,data)
get_by_id(
khata_id)



7.2 Sequence: User Views Home Screen (Khata List with Balances)

-------->|
khatas[] |
<---
--------|
| khatas[] |
|<--------------|
| for each |
| khata: |
| get_balance()|
|------------->|
SELECT SUM |
|----
------->|
{balance} |
<---
--------|
| balances[] |
|<--------------|
| KhataWithBalance |
| [] |
|<------------------|
| 200 OK + list |
|<--------------------|
| home screen |

7.3 Sequence: Settle a Khata




SELECT
khata
check owner
check OPEN
set SETTLED
UPDATE
OK
KhataResponse
200 OK
Khata moves
to Settled
tab

8. Dependency Injection Pattern

FastAPI uses a dependency injection system that makes it easy to provide services and repositories to route handlers without global state. This is how the layered architecture is wired together in practice.

python

```python
# app/dependencies.py
from fastapi import Depends
from sqlalchemy.ext.asyncio import AsyncSession
from app.database import get_session
from app.repositories.khata_repo import KhataRepository
from app.repositories.entry_repo import EntryRepository
from app.services.khata_service import KhataService
from app.services.entry_service import EntryService

# Each request gets its own database session (scoped to the request lifecycle)
async def get_khata_service(
    session: AsyncSession = Depends(get_session)
) -> KhataService:
    khata_repo = KhataRepository(session)
    party_repo = PartyRepository(session)
    return KhataService(khata_repo, party_repo)

async def get_entry_service(
    session: AsyncSession = Depends(get_session)
) -> EntryService:
    entry_repo = EntryRepository(session)
    khata_repo = KhataRepository(session)
    return EntryService(entry_repo, khata_repo)

# Usage in a route handler:
# @router.post("/khatas/{khata_id}/entries")
# async def add_entry(
#     khata_id: UUID,
#     data: EntryCreate,
#     service: EntryService = Depends(get_entry_service),
#     current_user: User = Depends(get_current_user)
# ):
#     entry = await service.add_entry(khata_id, data, current_user.id)
#     return EntryResponse.from_orm(entry)
```

End of LLD v1.0